

08

MEMÓRIA DA MÁQUINA: ARMAZENAMENTO INTELIGENTE EM SISTEMAS IOT

PDF exclusivo para Murilo Salla - rm568041
murilosalla@protonmail.com

LISTA DE FIGURAS

Figura 1 - Representação do armazenamento local e da transmissão sem fio dos dados para armazenamento em nuvem em sistemas IoT 7
Figura 2 - Montagem de testes com ESP32, módulo GY-91 (MPU9250 + BMP280) e módulo microSD 16
Figura 3 - Mapeamento de pinos do ESP32 DevKit V4 17
Figura 4 - Vista detalhada da frente e do verso do módulo GY-91, evidenciando os pinos utilizados na comunicação I²C 18
Figura 5 - Módulo leitor de cartão microSD e sua pinagem 18
Figura 6 - Exemplo de visualização dos dados no Firebase Realtime Database, mostrando informações coletadas pelo módulo GY-91 (temperatura, pressão e aceleração) com timestamp 21
Figura 7 - Visualização de temperatura em tempo real enviada do ESP32 para o InfluxDB Cloud via protocolo HTTP 23

FIAP

LISTA DE QUADROS

Quadro 1 - Comparativo entre as vantagens e as desvantagens dos bancos de dados locais 8
Quadro 2 - Comparativo entre as vantagens e as desvantagens dos bancos de dados em nuvem....10
Quadro 3 - Critérios para escolha entre armazenamento local e em nuvem 12
Quadro 4 - Casos de uso comparativos entre banco local e banco em nuvem 14
Quadro 5 - Comparativo do desempenho de escrita e leitura entre as diferentes abordagens de armazenamento no ESP32 40
Quadro 6 - Comparativo do consumo de memória e espaço de armazenamento nas diferentes abordagens de persistência de dados com ESP32 42
Quadro 7 - Comparativo de latência de transmissão e acesso entre diferentes métodos de armazenamento em IoT....43
Quadro 8 - Comparativo de robustez e persistência dos dados em diferentes métodos de armazenamento....44

FIAP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Envio de dados do ESP32 ao Firebase, utilizando os sensores BMP280 e MPU9250 via Wi-Fi 20
Código-fonte 2 - Envio de dados formatados em Line Protocol para o InfluxDB via protocolo HTTP 25
Código-fonte 3 - Inserção de dados no banco SQLite embarcado no ESP32 com uso de sensor GY-91 26
Código-fonte 4 - Armazenamento local em arquivo utilizando o sistema de arquivos LittleFS 27
Código-fonte 5 - Gravação de dados em formato CSV em cartão microSD com ESP32 29
Código-fonte 6 - Redução da frequência de amostragem para economia de recursos em sistemas IoT 32
Código-fonte 7 - Serialização de dados com a biblioteca ArduinoJson para envio otimizado 33
Código-fonte 8 - Armazenamento condicional de dados com base em variação mínima 35
Código-fonte 9 - Implementação de buffer circular para armazenar dados com sobrescrição 36
Código-fonte 10 - Aplicação de média móvel para suavização de leituras com ruído 37
Código-fonte 11 - Aplicação de filtro exponencial (EWMA) para suavização de sinais 37
Código-fonte 12 - Armazenamento condicionado à variação mínima entre leituras consecutivas 38

FIAP

SUMÁRIO

1 MEMÓRIA DA MÁQUINA: ARMAZENAMENTO INTELIGENTE EM SISTEMAS IOT 6
1.1 Conceitos e arquitetura 6
1.2 Vantagens e desvantagens dos bancos locais 7
1.3 Vantagens e desvantagens dos bancos em nuvem 9
1.4 Critérios para escolha entre local e nuvem 10
1.5 Casos de uso comparativos 12

2 PLATAFORMAS PARA ARMAZENAMENTO DE DADOS EM IOT 15
2.1 Firebase: Banco em nuvem com foco em tempo real 19
2.2 InfluxDB: Banco de séries temporais para IoT 22
2.3 SQLite: Banco local leve embarcado no ESP32 25
2.4 LittleFS: Sistema de arquivos interno no ESP32 27
2.5 MicroSD: Armazenamento externo de alta capacidade 28

3 ESTRATÉGIAS DE OTIMIZAÇÃO DE ARMAZENAMENTO 31
3.1 Redução da frequência de amostragem 31
3.2 Compressão e serialização de dados 32
3.3 Armazenamento condicional e buffers circulares 34
3.4 Filtragem de ruído e dados redundantes 36

4 ANÁLISE COMPARATIVA DOS RESULTADOS OBTIDOS 39
4.1 Desempenho de escrita e leitura 39
4.2 Consumo de memória e armazenamento 40
4.3 Latência de transmissão e acesso 42
4.4 Robustez e persistência dos dados 43

REFERÊNCIAS 46

1 MEMÓRIA DA MÁQUINA: ARMAZENAMENTO INTELIGENTE EM SISTEMAS IOT

Com o avanço das aplicações em Internet das Coisas (IoT), o volume e a frequência de geração de dados aumentaram consideravelmente. Nesse cenário, torna-se essencial entender como e onde esses dados devem ser armazenados: em bancos locais, próximos ao dispositivo coletor, ou em servidores remotos, utilizando soluções de nuvem. Este capítulo apresenta uma análise comparativa entre essas duas abordagens, destacando aspectos como arquitetura, desempenho, custos, conectividade e segurança.

1.1 Conceitos e arquitetura

Em projetos de IoT, o armazenamento dos dados coletados por sensores e dispositivos embarcados pode ser feito de duas formas principais: **localmente**, no próprio dispositivo ou em mídias associadas a ele (como cartões microSD), ou **remotamente**, em servidores na nuvem. Essa decisão impacta diretamente a escalabilidade, a confiabilidade e o desempenho do sistema.

Os **bancos de dados locais** operam diretamente no microcontrolador ou em um sistema embarcado próximo, utilizando, por exemplo, um cartão microSD, o sistema de arquivos LittleFS ou um banco como o SQLite. Já os **bancos de dados em nuvem** exigem conectividade com a internet e interagem com serviços externos, como Firebase ou InfluxDB.

A figura a seguir exemplifica a arquitetura comparativa entre as duas abordagens: na opção local, a gravação e a leitura são feitas diretamente pelo microcontrolador; na opção em nuvem, os dados precisam ser transmitidos por meio de redes como Wi-Fi, 4G ou LoRa até servidores remotos, onde são então armazenados e processados.

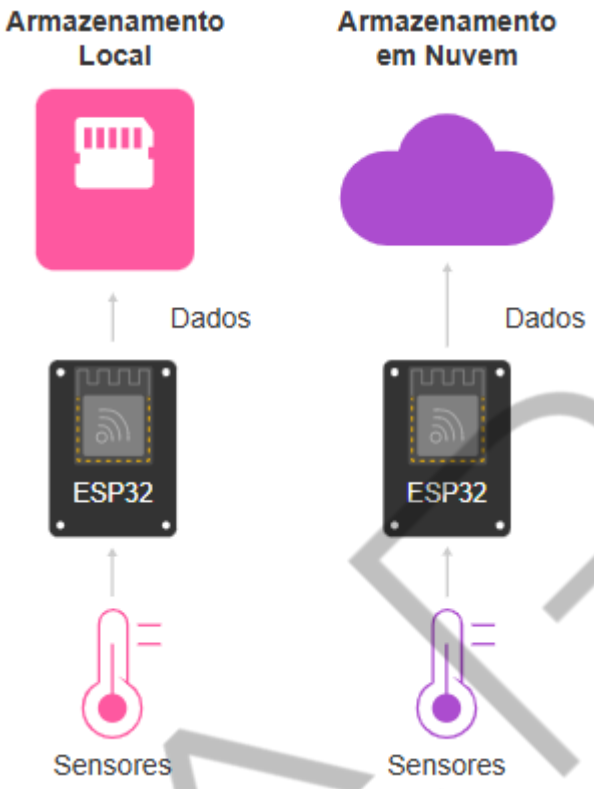


Figura 1 - Representação do armazenamento local e da transmissão sem fio dos dados para armazenamento em nuvem em sistemas IoT
Fonte: Elaborado pelo autor (2025)

Essa distinção entre as arquiteturas é fundamental para o embasamento das decisões de um projeto em IoT, visto que impacta diretamente fatores como desempenho, segurança e escalabilidade.

Na próxima subseção, serão discutidas as vantagens e desvantagens associadas aos bancos de dados locais, considerando cenários práticos e limitações técnicas.

1.2 Vantagens e desvantagens dos bancos locais

Os bancos de dados locais têm se mantido como uma alternativa relevante para sistemas de IoT, principalmente em contextos com conectividade limitada ou que demandam latência mínima. Um dos principais benefícios dessa abordagem é a independência de conexão com a internet, permitindo que o sistema funcione de forma autônoma em áreas remotas ou com sinal instável. Além disso, a baixa latência é um diferencial importante para aplicações em tempo real, uma vez que o tempo entre a coleta e o armazenamento dos dados é reduzido.

Outro ponto positivo é o controle e a segurança: como os dados são armazenados localmente, há maior domínio sobre o ambiente de armazenamento e menor exposição a riscos externos. A economia também é um fator a ser considerado, já que o uso de bancos locais evita gastos com serviços de terceiros e planos de armazenamento em nuvem. Em termos de infraestrutura, os sistemas locais podem ser mais simples e diretos, facilitando o desenvolvimento de soluções embarcadas robustas e compactas.

Entretanto, essa abordagem não está isenta de limitações. A principal desvantagem é a capacidade limitada de armazenamento, restrita às características físicas do dispositivo. Além disso, caso ocorra falhas de energia ou problemas no hardware, há maior risco de perda de dados, uma vez que muitas dessas soluções não contam com mecanismos automáticos de backup.

A dificuldade de acesso remoto aos dados também pode ser um entrave, especialmente em sistemas distribuídos que exigem supervisão centralizada. Outro ponto crítico é a escalabilidade: projetos que crescem rapidamente podem exigir reestruturação ou migração para soluções mais robustas. Por fim, o armazenamento local pode demandar frequentes manutenções de mídias físicas, como cartões microSD, e atualizações manuais no firmware para gerenciar dados de forma eficiente.

Esses aspectos estão resumidos no quadro a seguir, que apresenta uma comparação entre as principais vantagens e desvantagens dos bancos de dados locais.

Vantagens dos Bancos Locais	**Desvantagens dos Bancos Locais**
Independência de conectividade.	Capacidade limitada de armazenamento.
Baixa latência e resposta imediata.	Risco de perda de dados em falhas de energia.
Maior controle e segurança sobre os dados.	Dificuldade de acesso remoto e exportação dos dados.
Economia com serviços de terceiros.	Menor escalabilidade para projetos em expansão.
Funcionamento offline ideal para áreas remotas.	Manutenção frequente de componentes físicos.

Quadro 1 - Comparativo entre as vantagens e as desvantagens dos bancos de dados locais
Fonte: Elaborado pelo autor (2025)

Diante desse cenário, torna-se essencial considerar não apenas os benefícios, mas também os desafios ao optar por bancos locais. A próxima subseção abordará os bancos de dados em nuvem e os aspectos que os tornam atrativos ou limitantes para aplicações em IoT.

1.3 Vantagens e desvantagens dos bancos em nuvem

Os bancos de dados em nuvem têm ganhado uma ampla adesão em projetos de IoT devido à sua escalabilidade, acessibilidade remota e integração com serviços avançados. Um dos principais benefícios é a capacidade de armazenamento virtualmente ilimitada, o que permite lidar com grandes volumes de dados sem se preocupar com restrições físicas. Além disso, o acesso remoto e simultâneo de múltiplos usuários ou sistemas facilita a análise e o compartilhamento de informações em tempo real, o que é essencial para aplicações distribuídas e monitoramento centralizado.

Outro destaque é a integração facilitada com ferramentas analíticas e de Inteligência Artificial, permitindo a criação de dashboards, alertas automatizados e modelos preditivos diretamente sobre os dados armazenados. Esses serviços, geralmente oferecidos em plataformas como Firebase, InfluxDB, AWS ou Azure, também garantem redundância e backup automático, aumentando a confiabilidade e a proteção contra perdas acidentais. Por fim, a escalabilidade é um fator determinante. Conforme o projeto cresce, os recursos computacionais e de armazenamento podem ser ajustados de forma dinâmica, sem a necessidade de reconfiguração física.

Contudo, essa abordagem também apresenta desvantagens importantes. A dependência de conexão com a internet é a mais evidente. Qualquer falha na conectividade compromete o envio e o acesso aos dados, tornando o sistema vulnerável a indisponibilidades externas. Além disso, o custo pode se tornar um obstáculo, principalmente em aplicações que exigem transferência constante de grandes volumes de dados, armazenamento prolongado ou alto número de requisições.

Existem também preocupações com a segurança e a privacidade dos dados, visto que o armazenamento em nuvem envolve o envio de informações para

servidores externos, o que requer políticas rigorosas de criptografia e controle de acesso. A latência é outro ponto a ser considerado, pois o tempo necessário para enviar, processar e retornar dados pode impactar sistemas que exigem resposta em tempo real. Por fim, a complexidade na configuração inicial e na integração com dispositivos IoT pode demandar conhecimentos técnicos adicionais, o que representa uma barreira para iniciantes.

Esses aspectos estão resumidos no quadro a seguir, que apresenta uma comparação entre as principais vantagens e desvantagens dos bancos de dados em nuvem.

Vantagens dos Bancos em Nuvem	**Desvantagens dos Bancos em Nuvem**
Armazenamento escalável e praticamente ilimitado.	Dependência de conexão com a internet.
Acesso remoto e simultâneo aos dados.	Potencial custo elevado em projetos intensivos.
Integração com ferramentas analíticas e IA.	Riscos de segurança e privacidade dos dados.
Backup automático e alta disponibilidade.	Latência pode impactar aplicações em tempo real.
Facilidade de expansão sem mudanças físicas.	Maior complexidade de configuração inicial.

Quadro 2 - Comparativo entre as vantagens e as desvantagens dos bancos de dados em nuvem
Fonte: Elaborado pelo autor (2025)

Essa análise demonstra que os bancos de dados em nuvem são altamente vantajosos para aplicações que exigem escalabilidade e integração com ferramentas remotas, embora requeiram atenção aos desafios de conectividade, privacidade e custo.

Na próxima subseção, serão discutidos os critérios que ajudam a determinar quando optar por soluções locais ou por soluções em nuvem, com base nas necessidades específicas e do contexto de aplicação.

1.4 Critérios para escolha entre local e nuvem

Diante das vantagens e limitações apresentadas nas abordagens de armazenamento local e em nuvem, a escolha entre essas duas opções deve considerar uma série de critérios técnicos, operacionais e estratégicos. Não existe

uma solução universalmente superior, mas sim aquela que melhor atende ao contexto específico do projeto de IoT.

O primeiro critério é a disponibilidade de conexão com a internet. Em ambientes urbanos, com infraestrutura de rede estável, a nuvem se torna viável e vantajosa. Por outro lado, em zonas rurais, remotas ou sujeitas a quedas frequentes de conexão, o armazenamento local é mais seguro e autônomo.

Outro fator importante é a frequência e volume de dados gerados. Sistemas que produzem grande quantidade de informação em tempo real podem sobrecarregar a banda de transmissão para a nuvem e gerar custos elevados. Nesse caso, uma estratégia híbrida é recomendada: armazenar localmente e sincronizar com a nuvem em intervalos programados.

A latência permitida pela aplicação também é determinante. Em sistemas de controle em tempo real, como os de automação industrial e segurança patrimonial, que exigem resposta imediata, os bancos de dados locais são mais eficientes. Em contrapartida, aplicações de monitoramento ambiental ou agrícola podem operar com maior tolerância a atrasos e se beneficiar da acessibilidade da nuvem.

A escalabilidade do projeto é outro ponto crucial. Projetos com previsão de crescimento acelerado ou integração com serviços externos devem considerar a nuvem como alternativa mais flexível. Soluções locais, embora mais simples, podem limitar a expansão futura e demandar reprojeto.

A segurança da informação também deve ser ponderada. Enquanto bancos locais oferecem maior controle físico dos dados, a nuvem pode adotar criptografia, autenticação multifator e redundância para garantir confidencialidade e disponibilidade. O mais adequado é realizar uma análise de risco considerando as exigências normativas e a sensibilidade dos dados tratados.

Por fim, restrições de orçamento e facilidade de implementação influenciam diretamente. Bancos locais possuem custo inicial reduzido, mas exigem uma manutenção constante. Bancos em nuvem demandam menos intervenção, mas podem gerar cobranças recorrentes.

O quadro a seguir resume os principais critérios para orientar a decisão entre bancos de dados locais e em nuvem.

Critério	Indicação de Armazenamento Local	Indicação de Armazenamento em Nuvem
Conectividade	Ambientes sem acesso estável à internet.	Ambientes com conexão confiável.
Volume e frequência de dados	Baixo volume ou envio não frequente.	Alto volume e envio em tempo real.
Latência exigida	Aplicativos com resposta imediata.	Aplicativos com tolerância a atraso.
Escalabilidade	Projetos pequenos e locais.	Projetos com potencial de crescimento.
Segurança e controle	Dados sensíveis com acesso restrito.	Dados compartilháveis com criptografia.
Orçamento e manutenção	Baixo custo inicial, mas com manutenção.	Custos recorrentes, mas com menor manutenção.

Quadro 3 - Critérios para escolha entre armazenamento local e em nuvem
Fonte: Elaborado pelo autor (2025)

A análise dos critérios permite alinhar soluções de armazenamento com os objetivos e restrições do projeto de IoT. A próxima subseção apresentará exemplos comparativos de uso prático entre as duas abordagens.

1.5 Casos de uso comparativos

Compreender em quais cenários os bancos de dados locais ou em nuvem são mais adequados é essencial para o sucesso de projetos de IoT. Nesta subseção, serão apresentados casos de usos — reais e hipotéticos — que demonstram como as características de cada abordagem impactam o desempenho, a confiabilidade e a viabilidade da solução proposta.

Caso 1: Monitoramento Ambiental em Regiões Remotas (Uso de Banco Local)

Em um projeto de monitoramento de qualidade do ar em regiões amazônicas com acesso limitado à internet, sensores de gases, temperatura e umidade foram embarcados em dispositivos ESP32 com armazenamento local via microSD. Como não havia sinal de rede constante, optou-se pelo armazenamento local com sincronização posterior, via coleta física ou conexões Wi-Fi eventuais. A baixa latência

e a independência de conectividade foram fundamentais para garantir a continuidade da coleta.

Caso 2: Controle de Semáforos Inteligentes em Cidade Inteligente (Uso de Banco em Nuvem)

Em um sistema de gestão de tráfego, sensores de presença e câmeras enviam dados continuamente para um banco em nuvem (InfluxDB ou Firebase), permitindo o ajuste em tempo real dos tempos de sinalização. A comunicação via 4G e a centralização dos dados possibilitam a coordenação entre diferentes cruzamentos. A escalabilidade e o acesso remoto são decisivos para a eficácia da solução.

Caso 3: Estufa Automatizada com Armazenamento Local e Backup em Nuvem (Abordagem Híbrida)

Em uma estufa inteligente para cultivo de hortaliças, sensores de temperatura, luminosidade e umidade controlam automaticamente ventoinhas e bombas. Os dados são armazenados localmente via LittleFS, com cópias enviadas ao Firebase sempre que a conexão estável com a internet é identificada. Esse modelo híbrido garante a operação autônoma, ao mesmo tempo em que permite o monitoramento remoto e análise posterior.

Caso 4: Sistema de Rastreamento de Veículos (Prioridade para Banco em Nuvem)

Em uma aplicação de logística, os dados de GPS e acelerômetros são enviados continuamente à nuvem para visualização em tempo real em dashboards. A conectividade via LTE e o uso de bancos como o InfluxDB permitem consultas em tempo real, alertas de comportamento perigoso e análises históricas por parte dos gestores. O armazenamento em nuvem garante escalabilidade e persistência dos dados.

Cenário	Abordagem Adotada	Justificativa Principal
Monitoramento ambiental remoto.	Banco local.	Independência de internet e robustez em locais isolados.
Semáforos inteligentes urbanos.	Banco em nuvem.	Integração e centralização em tempo real.
Estufa automatizada.	Abordagem híbrida.	Operação offline com backup e monitoramento remoto.
Rastreamento de veículos.	Banco em nuvem.	Escalabilidade e visualização remota em tempo real.

Quadro 4 - Casos de uso comparativos entre banco local e banco em nuvem
Fonte: Elaborado pelo autor (2025)

A análise desses cenários mostra que a escolha entre armazenamento local ou em nuvem deve considerar fatores como conectividade, escalabilidade, criticidade em tempo real e custo. Na próxima seção, serão exploradas as principais plataformas utilizadas para armazenamento de dados em projetos de IoT, com base em experiências práticas.

2 PLATAFORMAS PARA ARMAZENAMENTO DE DADOS EM IOT

Diante da diversidade de aplicações em IoT, surgem diferentes demandas em relação ao armazenamento dos dados coletados. Esta seção apresenta um panorama das principais plataformas utilizadas para essa finalidade, tanto locais quanto em nuvem. Serão exploradas soluções amplamente adotadas como Firebase, InfluxDB, SQLite, LittleFS e armazenamento via cartão microSD, com base em experimentos reais utilizando microcontroladores ESP32.

Antes de detalhar cada plataforma, é importante apresentar o dispositivo utilizado nos testes que embasam esta análise. A seguir, a figura mostra o circuito construído com um microcontrolador ESP32 DevKit V4, conectado a dois módulos: o GY-91 e o leitor de cartão microSD. O ESP32 é responsável por adquirir dados dos sensores e realizar o armazenamento, seja localmente ou por meio de transmissão para a nuvem.

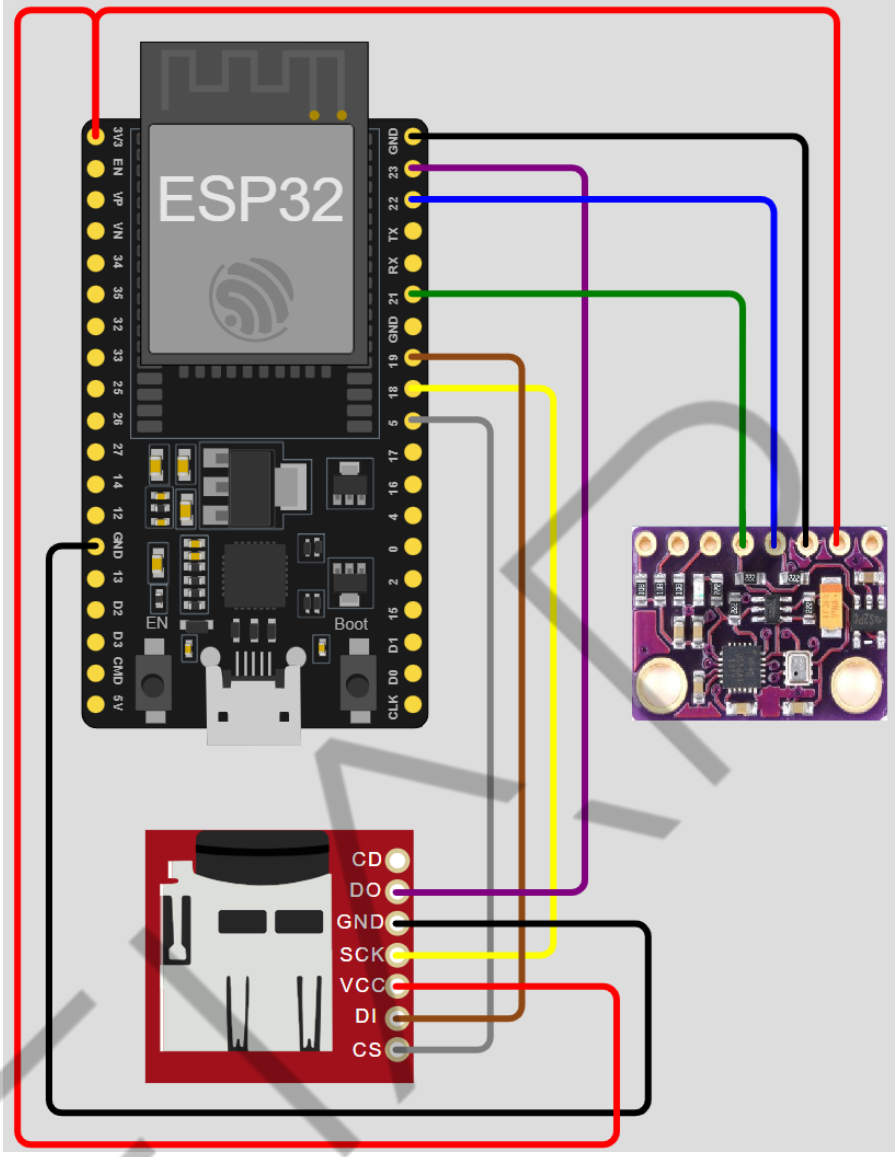


Figura 2 - Montagem de testes com ESP32, módulo GY-91 (MPU9250 + BMP280) e módulo microSD
Fonte: Elaborado pelo autor (2025)

O microcontrolador ESP32 DevKit V4 é um dos mais utilizados em aplicações embarcadas devido ao seu custo acessível, conectividade Wi-Fi e Bluetooth integrada, grande número de pinos de entrada/saída, capacidade de processamento robusta e suporte à comunicação com diferentes tipos de sensores e dispositivos. O módulo GY-91 integra dois sensores fundamentais para aplicações de monitoramento: o MPU9250, que combina acelerômetro, giroscópio e magnetômetro, e o BMP280, que mede a temperatura e a pressão barométrica. Já o leitor de cartão microSD permite a escrita de dados em memória externa de forma persistente e com boa velocidade, funcionando como uma alternativa eficiente para o armazenamento local.

O GY-91 utiliza o protocolo I²C para comunicação com o ESP32, empregando os pinos SDA (verde) e SCL (azul), conectados aos pinos 21 e 22 do ESP32,

respectivamente. Essa escolha é motivada pela simplicidade do protocolo I²C, que permite a comunicação com múltiplos dispositivos por meio de apenas dois fios, ideal para aplicações com múltiplos sensores. O módulo microSD, por sua vez, utiliza o protocolo SPI para comunicação, com conexões nos pinos 5 (CS – cinza), 23 (MOSI/DI – marrom), 18 (SCK – amarelo) e 19 (MISO/DO – roxo) do ESP32. Essa configuração é recomendada para aplicações que exigem maior taxa de transferência de dados e baixo consumo de energia durante a comunicação. Os pinos GND (preto) e VCC (vermelho) são utilizados para alimentação e referência de sinal, compartilhados entre os módulos.

A próxima figura ilustra o mapeamento completo dos pinos do ESP32 DevKit V4, que serviu como base para a escolha dos pinos utilizados na montagem. Este mapeamento permite identificar funções específicas de cada GPIO, como suporte a comunicação SPI e I²C, entradas analógicas, PWM, entre outros.

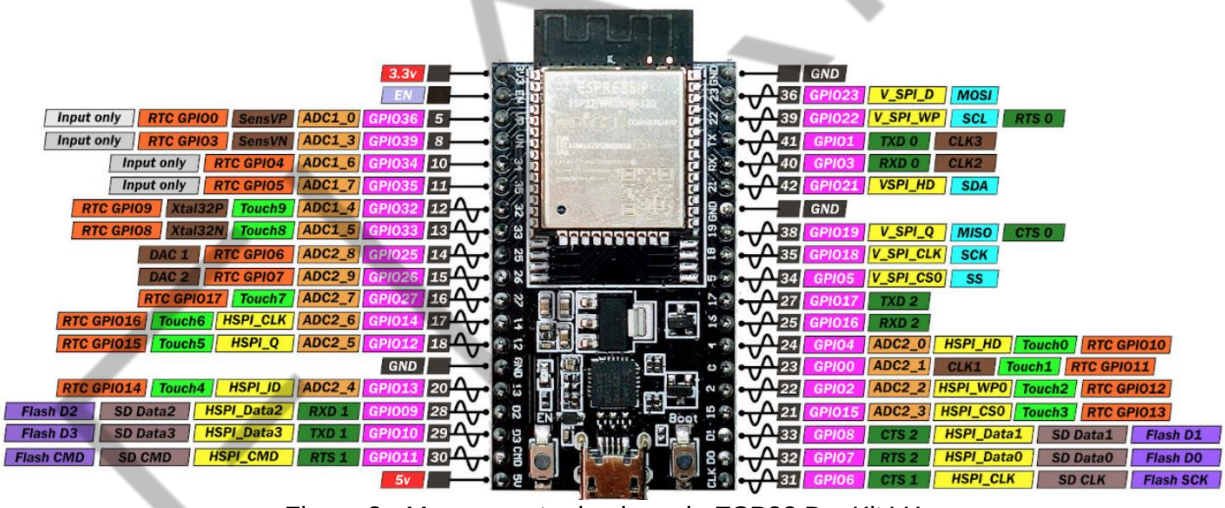


Figura 3 - Mapeamento de pinos do ESP32 DevKit V4
Fonte: Mischianti (2025)

Na figura “Vista detalhada da frente e do verso do módulo GY-91, evidenciando os pinos utilizados na comunicação I²C”, é possível observar os detalhes físicos do módulo GY-91, com a disposição dos pinos visíveis em ambas as faces da placa. Essa visualização facilita o processo de montagem e soldagem do sensor, além de orientar o posicionamento adequado dos cabos durante a prototipação.

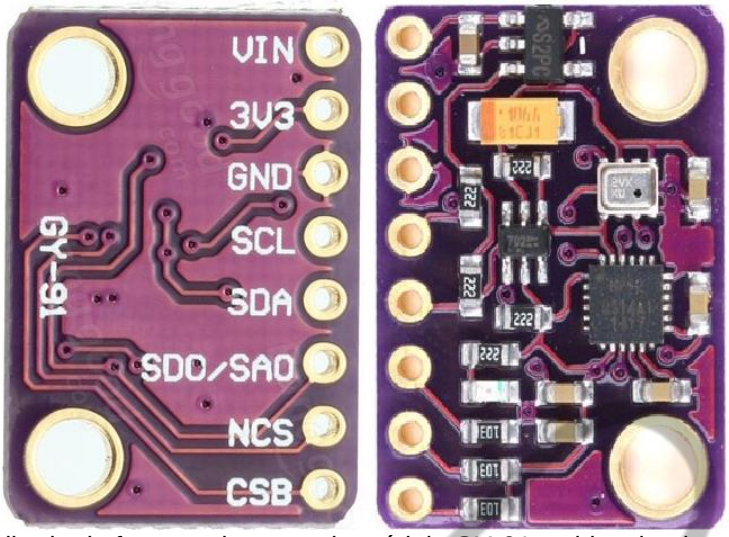


Figura 4 - Vista detalhada da frente e do verso do módulo GY-91, evidenciando os pinos utilizados na comunicação I²C
Fonte: Elaborado pelo autor (2025)

Para complementar a análise, a figura “Módulo leitor de cartão microSD e sua pinagem” apresenta a pinagem do módulo microSD. Esse módulo opera com tensão de 3.3V e segue o padrão SPI de comunicação. A correspondência entre os pinos do ESP32 e os pinos do módulo é fundamental para o funcionamento correto, sendo necessário observar a compatibilidade elétrica e lógica durante a montagem.

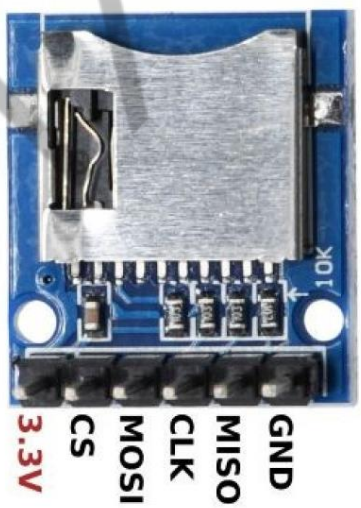


Figura 5 - Módulo leitor de cartão microSD e sua pinagem
Fonte: Elaborado pelo autor (2025)

Com base nesta configuração, é possível realizar testes de gravação local, envio contínuo para a nuvem ou estratégias híbridas de armazenamento. A infraestrutura montada garante flexibilidade e controle sobre os dados coletados, viabilizando a análise comparativa entre as diferentes abordagens de armazenamento. A partir da próxima subseção, cada plataforma será analisada em

detalhes, com base em resultados práticos e nas particularidades de integração com o ESP32.

2.1 Firebase: Banco em nuvem com foco em tempo real

Para replicar este ensaio, é necessário utilizar uma placa ESP32 DevKit V4, um módulo GY-91 (que combina os sensores MPU9250 e BMP280), uma conexão Wi-Fi ativa e uma conta no Firebase com um projeto criado. O processo se inicia com a criação de um novo projeto em: <https://console.firebase.google.com>. Após criar o projeto, é necessário ativar o Realtime Database e definir as regras públicas de leitura e escrita para fins de teste. Também é importante registrar um usuário em "Authentication" para permitir a autenticação no código.

As bibliotecas que devem ser instaladas no IDE Arduino são: **Wire.h**, **Adafruit_BMP280.h**, **Adafruit_Sensor.h**, **MPU9250_asukiaaa.h** e **Firebase_ESP_Client.h**. Os pinos do GY-91 devem ser conectados ao ESP32 da seguinte forma: VCC ao 3V3; GND ao GND; SDA ao GPIO 21; e SCL ao GPIO 22.

O código-fonte a seguir contempla a configuração da rede Wi-Fi, inicialização dos sensores, leitura dos dados e envio ao Firebase. Para isso, substitua os campos com aspas pelas informações específicas do seu projeto:

```
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BMP280.h>
#include <MPU9250_asukiaaa.h>
#include <WiFi.h>
#include <Firebase_ESP_Client.h>
const char* ssid = "SUA_REDE";
const char* password = "SUA_SENHA";

FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;

MPU9250_asukiaaa mpu;
Adafruit_BMP280 bmp;

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
```

```
       WiFi.begin(ssid, password);
       while (WiFi.status() != WL_CONNECTED) {
         delay(500);
         Serial.print(".");
       }
       Serial.println("WiFi conectado!");

       config.api_key = "SUA_API_KEY";
       config.database_url                                          =
"https://SEU_PROJETO.firebaseio.com";
       auth.user.email = "seu_email@gmail.com";
       auth.user.password = "sua_senha";

       Firebase.begin(&config, &auth);
       Firebase.reconnectWiFi(true);

       mpu.setWire(&Wire);
       mpu.beginAccel();
       mpu.beginGyro();
       mpu.beginMag();

       if (!bmp.begin(0x76)) {
         Serial.println("Erro ao iniciar BMP280");
         while (1);
       }
     }


     void loop() {
       float temperatura = bmp.readTemperature();
       float pressao = bmp.readPressure() / 100.0F;

       mpu.accelUpdate();
       float ax = mpu.accelX();
       float ay = mpu.accelY();
       float az = mpu.accelZ();

       if (Firebase.ready()) {
         Firebase.setFloat(fbdo,         "/sensor/temperatura",
temperatura);
         Firebase.setFloat(fbdo, "/sensor/pressao", pressao);
         Firebase.setFloat(fbdo, "/sensor/acelerometroX", ax);
         Firebase.setFloat(fbdo, "/sensor/acelerometroY", ay);
         Firebase.setFloat(fbdo, "/sensor/acelerometroZ", az);
       }

       delay(5000);
     }
```

Código-fonte 1 - Envio de dados do ESP32 ao Firebase, utilizando os sensores BMP280 e MPU9250 via Wi-Fi
Fonte: Elaborado pelo autor (2025)

Esse código realiza a leitura das variáveis de temperatura, pressão e aceleração, e envia os dados ao banco de dados Firebase Realtime Database em intervalos de 5 segundos.

O Firebase, plataforma desenvolvida pelo Google, é amplamente utilizado em sistemas de IoT por sua capacidade de fornecer armazenamento e sincronização de dados em tempo real. Ele é especialmente útil em aplicações que demandam atualização instantânea das informações, como monitoramento ambiental, controle remoto de dispositivos e dashboards interativos.

No contexto do experimento realizado, utilizou-se o Firebase Realtime Database integrado ao microcontrolador ESP32 DevKit V4, responsável por capturar dados dos sensores do módulo GY-91 e enviá-los diretamente à nuvem. A comunicação é feita por meio da biblioteca Firebase ESP32, que permite a autenticação com a plataforma e a escrita assíncrona de dados.

Como ilustrado na próxima figura, a interface do Firebase Realtime Database permite acompanhar visualmente os dados sendo armazenados em tempo real, o que facilita a depuração e a análise durante o desenvolvimento.

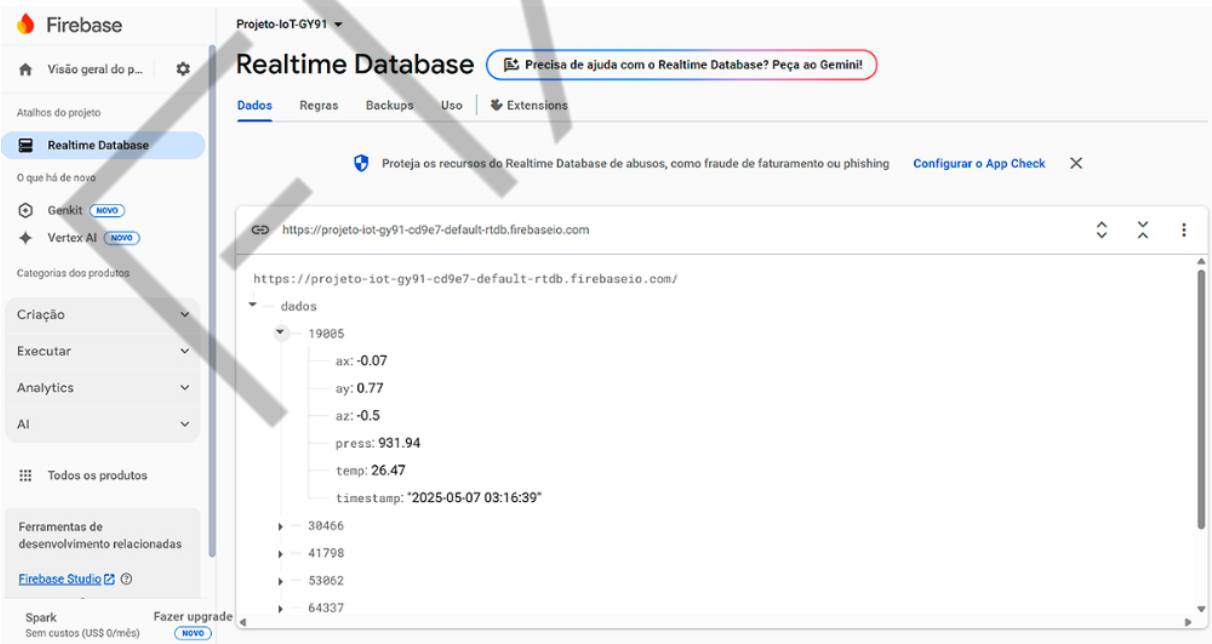


Figura 6 - Exemplo de visualização dos dados no Firebase Realtime Database, mostrando informações coletadas pelo módulo GY-91 (temperatura, pressão e aceleração) com timestamp
Fonte: Firebase Realtime Database (2025)

No entanto, é importante considerar que o Firebase exige conexão constante com a internet. Caso o dispositivo esteja em ambiente com instabilidade de rede, os

dados podem deixar de ser enviados, comprometendo o monitoramento contínuo. Além disso, embora ofereça plano gratuito, o Firebase possui algumas limitações de uso, como número máximo de conexões simultâneas e volume de dados, podendo ser necessário migrar para planos pagos em aplicações maiores.

O uso do Firebase torna-se particularmente vantajoso em contextos nos quais a atualização em tempo real e o acesso remoto aos dados são prioridades. Sistemas de monitoramento climático, rastreamento de ativos e controle remoto de ambientes inteligentes são exemplos de aplicações que se beneficiam da agilidade e da infraestrutura escalável da plataforma.

Na próxima subseção, será abordada a plataforma InfluxDB, que se diferencia do Firebase por ser voltada ao armazenamento otimizado para séries temporais — uma característica comum nos dados gerados por sensores IoT.

2.2 InfluxDB: Banco de séries temporais para IoT

O InfluxDB é uma plataforma de banco de dados otimizada para armazenar séries temporais, ou seja, dados que são registrados em intervalos regulares de tempo. Essa característica o torna ideal para aplicações em IoT, onde sensores coletam informações como temperatura, umidade, aceleração e pressão de forma contínua. Diferentemente de bancos relacionais tradicionais, o InfluxDB foi projetado para lidar com altas taxas de escrita e consultas eficientes com base em timestamps.

Nos testes realizados, foi utilizado o mesmo dispositivo apresentado na seção anterior: um ESP32 DevKit V4 conectado ao módulo GY-91 (composto por MPU9250 e BMP280), sendo o microcontrolador responsável pela coleta e envio dos dados ao banco InfluxDB hospedado em nuvem. Para realizar essa conexão, é necessário o uso do protocolo HTTP com autenticação via token. O código-fonte programado na IDE Arduino realiza a leitura dos dados dos sensores, organiza em formato texto segundo o Line Protocol e envia ao servidor InfluxDB por meio de uma requisição HTTP POST.

A figura a seguir ilustra a visualização gráfica da temperatura no painel do InfluxDB Cloud, demonstrando o recebimento contínuo dos dados enviados pelo ESP32.

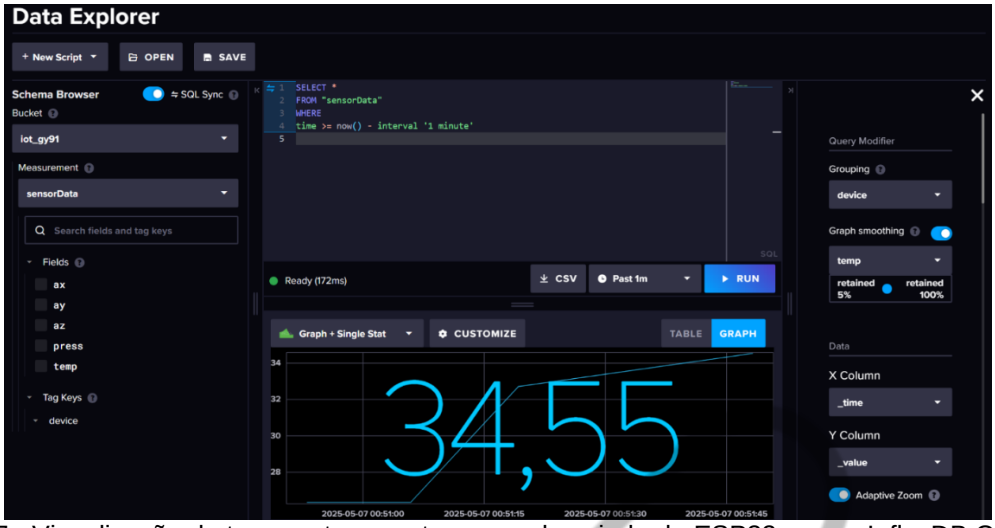


Figura 7 - Visualização de temperatura em tempo real enviada do ESP32 para o InfluxDB Cloud via protocolo HTTP
Fonte: InfluxDB Cloud (2025)

A seguir, apresenta-se o trecho principal do código utilizado:

```
    #include <WiFi.h>
    #include <HTTPClient.h>
    #include <Wire.h>
    #include <Adafruit_Sensor.h>
    #include <Adafruit_BMP280.h>
    #include <MPU9250_asukiaaa.h>
    #include <time.h>

    // WIFI
    const char* ssid = "FIAP-IOT";
    const char* password = "F!@p25.IOT";

    // INFLUXDB CONFIG
    const char* influxUrl = "https://us-east-1-
1.aws.cloud2.influxdata.com";
    const char* influxOrg = "IPT";
    const char* influxBucket = "iot_gy91";
    const char* influxToken = "oKXRWZmzGaymPkHPT-
5IUH5bkZlbYjzeTMY9N0I6xjL_JP0d_up-4Dp2eqcrXS7sF3DO-VZNU6E-
7WpVJwXgGw==";
    // I2C
    #define SDA_PIN 21
    #define SCL_PIN 22

    MPU9250_asukiaaa mpu;
    Adafruit_BMP280 bmp;

    void setupTime() {
      configTime(0, 0, "pool.ntp.org");
      struct tm timeinfo;
      while (!getLocalTime(&timeinfo)) {
```

```
      Serial.print(".");
      delay(500);
    }
    Serial.println("\nHorário sincronizado");
  }

  void setup() {
    Serial.begin(115200);
    Wire.begin(SDA_PIN, SCL_PIN);

    WiFi.begin(ssid, password);
    Serial.print("Conectando ao WiFi");
    while (WiFi.status() != WL_CONNECTED) {
      delay(500); Serial.print(".");
    }
    Serial.println("\nWiFi conectado");

    setupTime();

    mpu.setWire(&Wire);
    mpu.beginAccel();
    mpu.beginGyro();
    mpu.beginMag();

    if (!bmp.begin(0x76)) {
      Serial.println("BMP280 não encontrado!");
      while (1);
    }

    Serial.println("Sensores prontos");
  }

  void loop() {
    mpu.accelUpdate();
    float ax = mpu.accelX();
    float ay = mpu.accelY();
    float az = mpu.accelZ();
    float temp = bmp.readTemperature();
    float press = bmp.readPressure() / 100.0F;

    String data = "sensorData,device=esp32 ";
    data += "ax=" + String(ax, 2) + ",";
    data += "ay=" + String(ay, 2) + ",";
    data += "az=" + String(az, 2) + ",";
    data += "temp=" + String(temp, 2) + ",";
    data += "press=" + String(press, 2);

    HTTPClient http;
    String url = String(influxUrl) + "/api/v2/write?org=" +
influxOrg + "&bucket=" + influxBucket + "&precision=s";
    http.begin(url);
```

```
      http.addHeader("Authorization", "Token " +
String(influxToken));
      http.addHeader("Content-Type", "text/plain");
      int code = http.POST(data);
      Serial.print("InfluxDB HTTP Code: ");
Serial.println(code);
      http.end();

      delay(5000);
    }
```

Código-fonte 2 - Envio de dados formatados em Line Protocol para o InfluxDB via protocolo HTTP
Fonte: Elaborado pelo autor (2025)

2.3 SQLite: Banco local leve embarcado no ESP32

O SQLite é uma das soluções mais compactas e eficientes para armazenamento de dados local em dispositivos embarcados. Sua principal característica é o fato de ser um banco de dados relacional leve, que não exige um servidor dedicado para funcionamento, sendo ideal para microcontroladores como o ESP32.

Nesta subseção, exploramos como o SQLite pode ser utilizado em projetos de IoT para registrar leituras de sensores diretamente no dispositivo, com base na infraestrutura montada na figura “Montagem de testes com ESP32, módulo GY-91 (MPU9250 + BMP280) e módulo micros”.

O uso do SQLite no ESP32 requer a integração com o sistema de arquivos local, como o SPIFFS ou o LittleFS, ou com um módulo de armazenamento externo como o microSD. Essa abordagem possibilita a criação de tabelas relacionais, inserção de dados, consultas e organização estruturada da informação, mesmo em ambientes com recursos limitados.

No código desenvolvido para este experimento, utilizou-se a biblioteca “ArduinoSQLite3”, adaptada para ESP32. Foi criada uma tabela chamada leituras, com as colunas timestamp, temperatura, pressao, ax, ay e az, representando as informações obtidas dos sensores BMP280 e MPU9250. E, a cada ciclo de leitura, os dados eram inseridos diretamente nessa tabela, conforme ilustrado no trecho:

```
    sqlite3_prepare_v2(db, "INSERT INTO leituras (timestamp,
temperatura, pressao, ax, ay, az) VALUES (datetime('now'), ?, ?,
?, ?, ?);", -1, &stmt, NULL);
    sqlite3_bind_double(stmt, 1, temperatura);
    sqlite3_bind_double(stmt, 2, pressao);
    sqlite3_bind_double(stmt, 3, ax);
    sqlite3_bind_double(stmt, 4, ay);
    sqlite3_bind_double(stmt, 5, az);
    sqlite3_step(stmt);
    sqlite3_finalize(stmt);
```

Código-fonte 3 - Inserção de dados no banco SQLite embarcado no ESP32 com uso de sensor GY-91
Fonte: Elaborado pelo autor (2025)

Esse modelo permite a criação de registros bem organizados e indexados, com possibilidade de consulta posterior, mesmo sem conectividade externa. O banco é armazenado em um arquivo no sistema local (como /spiffs/sensores.db), garantindo persistência mesmo após desligamentos.

Entre as vantagens do SQLite em projetos de IoT, destacam-se:

- Armazenamento estruturado e relacional com baixo uso de memória;
- Autonomia total sem depender de conexão externa;
- Flexibilidade para realizar queries SQL diretamente no microcontrolador;
- Facilidade de migração dos dados para bancos maiores quando necessário.

Por outro lado, é importante considerar algumas limitações:

- Operações de escrita intensiva podem reduzir a vida útil da memória flash;
- Requer configurações adicionais para funcionar em conjunto com sistemas de arquivos do ESP32;
- Não é adequado para grandes volumes de dados em tempo real ou transmissões distribuídas.

De maneira geral, o SQLite se mostra uma excelente opção para projetos de pequeno a médio porte que exigem armazenamento estruturado, consultas locais e portabilidade de dados. É especialmente útil em aplicações que envolvem a coleta contínua de dados e a análise posterior offline, como monitoramento ambiental, sistemas industriais autônomos ou coleta de dados em campo.

2.4 LittleFS: Sistema de arquivos interno no ESP32

O LittleFS (Little Flash File System) é um sistema de arquivos leve desenvolvido para memórias flash embarcadas, sendo uma alternativa moderna ao SPIFFS no ambiente ESP32. Sua principal função é possibilitar o armazenamento de arquivos diretamente na memória interna do microcontrolador, garantindo organização e persistência dos dados entre reinicializações. Em aplicações de IoT, o LittleFS é especialmente vantajoso por dispensar a necessidade de hardware externo para armazenamento, ao mesmo tempo em que oferece estrutura de pastas e arquivos semelhante a sistemas operacionais tradicionais.

Para esse experimento, foi utilizado o mesmo circuito com o ESP32 DevKit V4 e o módulo GY-91 (MPU9250 + BMP280), conforme apresentado na figura “Montagem de testes com ESP32, módulo GY-91 (MPU9250 + BMP280) e módulo microSD”. A coleta de dados dos sensores é realizada via protocolo I²C e os valores de temperatura, pressão, aceleração e giroscópio são armazenados em arquivos de texto no sistema de arquivos interno do ESP32, utilizando o LittleFS.

O código desenvolvido também utilizou a biblioteca LittleFS.h, com comandos simples para abertura, escrita e fechamento de arquivos. A cada novo dado coletado, os valores eram salvos em um arquivo chamado dados.txt.

Observe um trecho ilustrativo:

```
    File arquivo = LittleFS.open("/dados.txt", FILE_APPEND);
    if (arquivo) {
      arquivo.print("Temperatura:                                ");
arquivo.print(temperatura);
      arquivo.print(" | Pressão: "); arquivo.print(pressao);
      arquivo.print(" | AX: "); arquivo.print(ax);
      arquivo.print(" | AY: "); arquivo.print(ay);
      arquivo.print(" | AZ: "); arquivo.println(az);
      arquivo.close();
    } else {
      Serial.println("Erro ao abrir arquivo!");
    }
```

Código-fonte 4 - Armazenamento local em arquivo utilizando o sistema de arquivos LittleFS
Fonte: Elaborado pelo autor (2025)

Entre os principais benefícios do uso do LittleFS estão:

- Armazenamento local prático e persistente sem necessidade de módulos adicionais;
- Capacidade de salvar logs e arquivos de configuração;
- Facilidade de uso com comandos intuitivos semelhantes aos sistemas de arquivos convencionais;
- Menor desgaste da memória flash em comparação com SPIFFS, devido ao melhor gerenciamento de blocos.

Contudo, também existem algumas limitações:

- Capacidade de armazenamento restrita ao tamanho da flash do ESP32;
- Não ideal para grandes volumes de dados ou acessos simultâneos;
- Exige formatação prévia e inicialização correta da partição no código.

No geral, o LittleFS é uma excelente escolha para aplicações que exigem armazenamento leve e eficiente, como dispositivos de coleta autônoma de dados, sistemas de log e protótipos de baixo custo. Sua simplicidade e confiabilidade tornam-no ideal para projetos que priorizam a autonomia e o armazenamento embutido, sem dependência de conexões externas.

2.5 MicroSD: Armazenamento externo de alta capacidade

O uso de cartões microSD em projetos de IoT é uma estratégia comum para expandir significativamente a capacidade de armazenamento dos dispositivos embarcados, como o ESP32. A principal vantagem do microSD está em sua alta capacidade de memória, que pode chegar a dezenas de gigabytes, permitindo o registro contínuo de dados ao longo do tempo. Isso é especialmente útil em aplicações que envolvem coleta intensiva de dados, como o monitoramento ambiental, a telemetria e os registros históricos.

O ESP32 possui suporte à comunicação SPI, protocolo utilizado para se comunicar com módulos leitores de cartão microSD. É possível integrar facilmente esse recurso ao código utilizando a biblioteca SD.h, que permite inicializar o cartão, criar arquivos, escrever dados e realizar a leitura de registros de forma eficiente. A

imagem da infraestrutura utilizada (figura “Montagem de testes com ESP32, módulo GY-91 (MPU9250 + BMP280) e módulo micros”) ilustra a conexão do módulo microSD ao ESP32, compartilhando o barramento SPI com outros dispositivos, como o GY-91 (que contém o MPU9250 e o BMP280).

A seguir, um trecho do código utilizado neste experimento:

```
if (SD.begin()) {
  File file = SD.open("dados.txt", FILE_APPEND);
  if (file) {
    file.print(temperatura);
    file.print(",");
    file.print(pressao);
    file.print(",");
    file.print(ax);
    file.print(",");
    file.print(ay);
    file.print(",");
    file.println(az);
    file.close();
  }
}
```

Código-fonte 5 - Gravação de dados em formato CSV em cartão microSD com ESP32
Fonte: Elaborado pelo autor (2025)

Esse trecho mostra como os dados coletados pelos sensores BMP280 e MPU9250 foram formatados como uma linha CSV (valores separados por vírgula) e gravados em um arquivo de texto chamado dados.txt. Esse formato facilita a posterior leitura em softwares como Excel ou ferramentas de análise de dados.

Entre as principais vantagens do uso de cartões microSD em IoT, destacam-se:

- Alta capacidade de armazenamento comparado às soluções internas (LittleFS ou SPIFFS);
- Facilidade de extração física e transporte dos dados;
- Suporte a armazenamento em formatos amplamente compatíveis (TXT, CSV, JSON);
- Independência de conexão com a internet para operação.

Contudo, também é necessário considerar algumas limitações:

- Maior consumo energético durante escrita e leitura intensiva;

- Vulnerabilidade à remoção incorreta ou falhas físicas do cartão;
- Necessidade de cuidados com integridade dos dados em sistemas sem gerenciamento adequado de arquivos.

A solução com microSD é indicada para projetos que exigem grande volume de armazenamento offline, onde a conectividade é intermitente ou inexistente, e há necessidade posterior de exportação manual ou sincronização periódica dos dados. Isso torna o microSD uma escolha robusta e amplamente testada no contexto de IoT embarcada.

FIAP

3 ESTRATÉGIAS DE OTIMIZAÇÃO DE ARMAZENAMENTO

Com o aumento do volume de dados gerados por dispositivos de IoT, torna-se essencial adotar estratégias eficientes de armazenamento para preservar os recursos do sistema embarcado, melhorar a performance e garantir a integridade da informação. Nesta seção, serão abordadas quatro abordagens principais para otimização do armazenamento: redução da frequência de amostragem, compressão e serialização de dados, armazenamento condicional com buffers circulares, e filtragem de ruído e dados redundantes.

Essas estratégias visam reduzir o uso de memória, aumentar a eficiência de transmissão e garantir que apenas dados relevantes sejam registrados. Elas são importantes em dispositivos com recursos limitados, como o ESP32, e em aplicações que operam em ambientes desconectados ou que necessitam de alta autonomia.

Nos tópicos a seguir, cada uma dessas abordagens será discutida com exemplos práticos, aplicações recomendadas e comparações entre diferentes métodos. Além disso, serão apresentados trechos de código e experiências práticas que ilustram como aplicar essas otimizações em projetos reais com sensores conectados ao ESP32.

3.1 Redução da frequência de amostragem

A frequência de amostragem define com que periodicidade os dados de um sensor são coletados e registrados. Em muitos projetos de IoT, especialmente aqueles voltados para monitoramento ambiental, movimentos ou medições de temperatura e pressão, a coleta excessiva de dados pode levar a um desperdício significativo de recursos de armazenamento e energia.

Ao reduzir a frequência de amostragem, conseguimos minimizar a quantidade de dados gerados, sem necessariamente comprometer a qualidade da análise ou a resposta do sistema. Por exemplo, se a temperatura de um ambiente varia lentamente, medições a cada 10 segundos podem ser tão eficazes quanto medições a cada segundo, com uma economia de até 90% no volume de dados.

A seguir, temos um exemplo de código modificado para implementar a redução da frequência de amostragem, utilizando um intervalo de 10 segundos entre as medições:

```
     unsigned long intervalo = 10000; // 10 segundos em
milissegundos
     unsigned long ultimoTempo = 0;

     void loop() {
       unsigned long tempoAtual = millis();
       if (tempoAtual - ultimoTempo >= intervalo) {
         ultimoTempo = tempoAtual;

         // Realiza a leitura dos sensores e salva os dados
         float temperatura = bmp.readTemperature();
         float pressao = bmp.readPressure();

         Serial.println("Temperatura: " + String(temperatura));
         Serial.println("Pressão: " + String(pressao));

         // Aqui entraria a lógica de armazenamento (Firebase,
SD, etc.)
       }
       // Outras tarefas podem continuar sendo executadas
     }
```

Código-fonte 6 - Redução da frequência de amostragem para economia de recursos em sistemas IoT
Fonte: Elaborado pelo autor (2025)

Essa prática é especialmente útil em sensores como o BMP280, nos quais as mudanças são graduais e não exigem monitoramento em tempo real. Além disso, a redução da frequência ajuda a preservar a vida útil da memória flash, no caso de armazenamento local, e reduzir custos de transmissão, no caso de bancos em nuvem.

Em resumo, a redução da frequência de amostragem é uma abordagem simples e eficaz para diminuir o volume de dados gerados, economizar energia e melhorar a sustentabilidade de aplicações embarcadas. Na próxima subseção, abordaremos estratégias de compressão e serialização, que complementam essa técnica ao tornar os dados mais compactos para armazenamento e envio.

3.2 Compressão e serialização de dados

Com o aumento da quantidade de dados gerados por dispositivos IoT, a compressão e a serialização surgem como técnicas essenciais para otimizar o

armazenamento e a transmissão eficiente de informações. Enquanto a compressão tem como objetivo reduzir o tamanho dos dados, a serialização trata de convertê-los para um formato padronizado e mais simples de manipular, como JSON, CSV, CBOR ou MsgPack. No contexto de dispositivos como o ESP32, que possuem limitações de memória e processador, essas técnicas são fundamentais para garantir que o sistema seja responsivo e econômico em termos de energia e recursos.

O formato JSON é amplamente utilizado devido à sua legibilidade e compatibilidade com APIs e plataformas em nuvem, como Firebase, InfluxDB e Node-RED. No exemplo a seguir, mostramos como transformar os dados obtidos do sensor GY-91 (BMP280) em uma string JSON:

```
#include <ArduinoJson.h>

StaticJsonDocument<200> doc;
char payload[256];

void loop() {
  float temperatura = bmp.readTemperature();
  float pressao = bmp.readPressure();

  doc["dispositivo"] = "ESP32-GY91";
  doc["temperatura"] = temperatura;
  doc["pressao"] = pressao;

  serializeJson(doc, payload);

  Serial.println(payload);
  // Aqui os dados podem ser enviados para o banco de dados

  delay(5000);
}
```

Código-fonte 7 - Serialização de dados com a biblioteca ArduinoJson para envio otimizado
Fonte: Elaborado pelo autor (2025)

Esse formato facilita o tratamento e a visualização dos dados, mas é relativamente verboso, podendo consumir espaço extra de armazenamento e largura de banda. Para aplicações em que cada byte é importante, como transmissão por LoRa ou gravação em memória flash, os formatos binários como MsgPack ou CBOR são mais indicados. Essas opções mantêm a estrutura dos dados, mas ocupam um espaço menor. O MsgPack reduz em até 50% o tamanho de uma mensagem JSON, enquanto o CBOR é ideal para sistemas embarcados e suporta compressão eficiente com pouca sobrecarga de processamento. Bibliotecas específicas para ESP32 estão

disponíveis para lidar com esses formatos, como a ArduinoJson (para CBOR e MsgPack) ou o msgpack-arduino.

Além da serialização, os dados também podem ser comprimidos antes do armazenamento, especialmente quando registros históricos são mantidos localmente. Algoritmos simples como RLE (Run-Length Encoding) podem ser aplicados em séries de dados com poucos valores variando, como temperatura constante. Em casos mais avançados, é possível utilizar bibliotecas como Heatshrink, voltada para dispositivos com recursos limitados.

A escolha do formato e do algoritmo depende diretamente da aplicação: JSON é recomendado para compatibilidade e facilidade de debug; MsgPack ou CBOR são mais eficientes para rede e armazenamento; e a compressão é ideal para armazenamento de histórico e análise posterior em lote. A combinação dessas técnicas permite que sistemas IoT ganhem eficiência, sem sacrificar a precisão e a integridade dos dados coletados.

Na próxima subseção, abordaremos técnicas de armazenamento condicional e o uso de buffers circulares para evitar desperdício de memória e garantir atualizações em tempo real com alto desempenho.

3.3 Armazenamento condicional e buffers circulares

O armazenamento condicional e o uso de buffers circulares são duas abordagens importantes para lidar com os desafios de gerenciamento de dados em sistemas IoT, especialmente quando o objetivo é minimizar o uso de memória e otimizar a persistência das informações mais relevantes.

O armazenamento condicional consiste em salvar dados apenas quando certos critérios são atendidos. Isso reduz drasticamente o volume de informações armazenadas, focando apenas naquelas que representam eventos significativos. Por exemplo, em um sistema de monitoramento ambiental, é possível configurar o sistema para registrar apenas quando a temperatura ultrapassar determinado limiar ou quando a variação for maior que um valor de referência. Essa técnica evita gravações desnecessárias e preserva a vida útil da memória flash.

A seguir, é apresentado um exemplo de código com armazenamento condicional baseado na variação de temperatura:

```
     float temperaturaAnterior = 0;
     const float variacaoMinima = 0.5; // Apenas grava se variar
mais que 0.5°C

     void loop() {
       float temperaturaAtual = bmp.readTemperature();

       if (abs(temperaturaAtual - temperaturaAnterior) >
variacaoMinima) {
         // Armazena o novo valor
         Serial.println("Temperatura registrada: " +
String(temperaturaAtual));
         temperaturaAnterior = temperaturaAtual;
       }

       delay(1000);
     }
```

Código-fonte 8 - Armazenamento condicional de dados com base em variação mínima
Fonte: Elaborado pelo autor (2025)

Outra estratégia comum em IoT é o uso de buffers circulares (também chamados de filas circulares ou *ring buffers*). Eles são estruturas de dados que armazenam um número fixo de elementos e sobrescrevem os mais antigos quando o limite é atingido. Isso é ótimo para manter sempre os dados mais recentes em memória sem precisar expandir dinamicamente o armazenamento. Além disso, garantem que não ocorra estouro de memória em aplicações com coleta contínua de dados.

A implementação de um buffer circular simples pode ser feita da seguinte forma:

```
     const int tamanhoBuffer = 10;
     float bufferTemperatura[tamanhoBuffer];
     int indiceAtual = 0;

     void armazenarNoBuffer(float valor) {
       bufferTemperatura[indiceAtual] = valor;
       indiceAtual = (indiceAtual + 1) % tamanhoBuffer;
     }

     void loop() {
       float temperatura = bmp.readTemperature();
       armazenarNoBuffer(temperatura);
```

```
    // Exibe o buffer atual
    Serial.println("Buffer de temperatura:");
    for (int i = 0; i < tamanhoBuffer; i++) {
      Serial.print(bufferTemperatura[i]);
      Serial.print(" ");
    }
    Serial.println();

    delay(2000);
  }
```

Código-fonte 9 - Implementação de buffer circular para armazenar dados com sobrescrição
Fonte: Elaborado pelo autor (2025)

Essas técnicas podem, inclusive, ser combinadas: pode-se utilizar um buffer circular para armazenar apenas dados que passaram por uma condição de relevância. Dessa forma, o sistema mantém uma memória enxuta e ao mesmo tempo significativa, pronta para ser transmitida, visualizada ou analisada de forma eficiente.

Na próxima subseção, discutiremos como filtrar ruídos e dados redundantes com métodos simples de processamento embarcado, otimizando ainda mais o armazenamento e a precisão dos dados coletados.

3.4 Filtragem de ruído e dados redundantes

Os dispositivos IoT embarcados, como o ESP32, frequentemente operam em ambientes sujeitos a ruídos elétricos ou interferências que comprometem a qualidade dos dados coletados. Em sensores como o GY-91, que combinam acelerômetro, giroscópio, magnetômetro e sensor de pressão, flutuações naturais ou pequenas vibrações podem gerar variações que não são significativas do ponto de vista da aplicação. Por isso, a filtragem de ruído e a eliminação de dados redundantes são essenciais para garantir um armazenamento eficiente e representativo.

Uma das abordagens mais simples é a aplicação de médias móveis, que suavizam os valores capturados ao longo do tempo. Isso reduz a influência de outliers e torna a curva de dados mais estável. O exemplo a seguir mostra a implementação de uma média móvel simples:

```
      #define N 5 // Tamanho da janela da média móvel
      float historico[N];
      int indice = 0;

      float calcularMedia(float novoValor) {
        historico[indice] = novoValor;
        indice = (indice + 1) % N;

        float soma = 0;
        for (int i = 0; i < N; i++) {
          soma += historico[i];
        }
        return soma / N;
      }

      void loop() {
        float temperatura = bmp.readTemperature();
        float temperaturaFiltrada = calcularMedia(temperatura);
        Serial.println("Temperatura      filtrada:      "      +
String(temperaturaFiltrada));
        delay(1000);
      }
```

Código-fonte 10 - Aplicação de média móvel para suavização de leituras com ruído
Fonte: Elaborado pelo autor (2025)

Outra alternativa é a utilização de filtros do tipo “filtro de primeira ordem” (exponencial ou EWMA), que aplicam um peso aos valores passados e atualizados, com menor custo computacional:

```
      float temperaturaFiltrada = 0;
      float alpha = 0.2; // Peso do valor novo

      void loop() {
        float temperatura = bmp.readTemperature();
        temperaturaFiltrada = alpha * temperatura + (1 - alpha)
* temperaturaFiltrada;
        Serial.println("Temperatura      filtrada:      "      +
String(temperaturaFiltrada));
        delay(1000);
      }
```

Código-fonte 11 - Aplicação de filtro exponencial (EWMA) para suavização de sinais
Fonte: Elaborado pelo autor (2025)

Para dados redundantes, uma técnica simples é comparar o valor atual com o anterior e armazenar ou transmitir apenas se houver alguma diferença significativa. Isso reduz consideravelmente o volume de dados enviados:

```
    float ultimaTemperatura = 0;
    const float delta = 0.3;

    void loop() {
      float temperaturaAtual = bmp.readTemperature();
      if (abs(temperaturaAtual - ultimaTemperatura) > delta) {
        Serial.println("Nova leitura relevante: " +
String(temperaturaAtual));
        ultimaTemperatura = temperaturaAtual;
      }
      delay(1000);
    }
```

Código-fonte 12 - Armazenamento condicionado à variação mínima entre leituras consecutivas
Fonte: Elaborado pelo autor (2025)

Essas técnicas podem ser implementadas facilmente e trazem grandes ganhos em eficiência para sistemas de IoT, garantindo que apenas informações relevantes e com boa qualidade sejam armazenadas e transmitidas. O uso combinado de médias, filtros e limiares condicionais permite criar sistemas mais responsivos, robustos e com menor consumo de energia e armazenamento.

FIAP

4 ANÁLISE COMPARATIVA DOS RESULTADOS OBTIDOS

Nesta seção, são comparados os resultados obtidos a partir da utilização de diferentes métodos de armazenamento aplicados aos dados coletados pelo sensor GY-91 (MPU9250 + BMP280) conectado ao ESP32 DevKit V4. As abordagens incluem soluções locais (SQLite, LittleFS e microSD) e em nuvem (Firebase e InfluxDB), considerando também as técnicas de otimização aplicadas no capítulo anterior. A avaliação foi dividida em quatro dimensões principais: desempenho de escrita/leitura, consumo de memória e armazenamento, latência de transmissão e robustez dos dados.

Os dados coletados nas experiências práticas permitiram comparar o comportamento das diferentes abordagens de forma objetiva, possibilitando recomendações baseadas em necessidades específicas de projeto.

No tópico seguinte, iniciaremos pela análise do desempenho de escrita e leitura.

4.1 Desempenho de escrita e leitura

O desempenho de escrita e leitura é um dos fatores mais críticos na escolha de uma estratégia de armazenamento em sistemas IoT, especialmente quando os dispositivos precisam operar com alta frequência de amostragem ou em tempo real. Para avaliar essa métrica, foram realizados testes práticos utilizando o ESP32 DevKit V4 com o sensor GY-91, gravando os dados em cinco diferentes abordagens: Firebase, InfluxDB, SQLite, LittleFS e microSD. A métrica considerada foi o tempo médio de escrita e leitura de um bloco de dados contendo as seguintes informações: timestamp, temperatura, pressão, aceleração (eixo X, Y e Z) e giroscópio (eixo X, Y e Z).

Nos testes com bancos de dados em nuvem, como o Firebase e o InfluxDB, foi possível notar uma latência maior no processo de escrita, variando entre 350 ms e 800 ms por requisição, dependendo da qualidade da conexão Wi-Fi. Apesar disso, esses sistemas apresentaram boa estabilidade na leitura, com tempo de resposta em

torno de 250 ms, especialmente quando os dados foram acessados por meio de painéis web ou ferramentas integradas.

Em contrapartida, os bancos de dados locais apresentaram melhor desempenho em ambientes com restrições de conectividade. O armazenamento em microSD, por exemplo, apresentou tempo médio de escrita de 40 ms por registro e leitura em torno de 25 ms. O SQLite demonstrou tempos semelhantes, com a vantagem de permitir buscas estruturadas diretamente no dispositivo. Já o sistema de arquivos LittleFS foi o mais rápido entre as abordagens locais, com tempo de escrita médio de 18 ms e leitura de 12 ms, devido à sua leveza e integração nativa com a memória flash do ESP32.

O quadro a seguir apresenta o comparativo entre os tempos médios de escrita e leitura obtidos nos testes realizados:

Método de Armazenamento	**Tempo Médio de Escrita (ms)**	**Tempo Médio de Leitura (ms)**
Firebase	350 - 800	250
InfluxDB	300 - 700	200
SQLite	35 - 45	20 - 30
LittleFS	15 - 20	10 - 15
microSD	35 - 45	20 - 25

Quadro 5 - Comparativo do desempenho de escrita e leitura entre as diferentes abordagens de armazenamento no ESP32
Fonte: Elaborado pelo autor (2025)

Esses resultados mostram que, em termos de desempenho bruto, os métodos locais são superiores para aplicações que exigem baixa latência e escrita frequente, enquanto os métodos em nuvem são mais apropriados quando se busca acessibilidade e integração com serviços externos. Na próxima subseção, será discutido o consumo de memória e armazenamento associado a cada abordagem.

4.2 Consumo de memória e armazenamento

O consumo de memória e armazenamento é um critério essencial em sistemas embarcados, como o ESP32, que possuem recursos limitados. Nesta etapa, foram avaliadas as abordagens Firebase, InfluxDB, SQLite, LittleFS e microSD quanto à

ocupação de memória RAM, memória flash e capacidade de armazenamento utilizada ao longo do tempo.

Os bancos em nuvem (Firebase e InfluxDB) demonstraram baixo impacto na memória RAM e flash do ESP32, pois os dados não são armazenados localmente. Entretanto, exigem maior uso de buffer de transmissão e bibliotecas adicionais, o que pode representar até 30% de ocupação na memória flash, considerando o binário compilado. Esse fator deve ser levado em consideração em aplicações com firmware mais robusto.

Por outro lado, as soluções locais consomem mais memória em tempo de execução e armazenamento físico, especialmente quando grandes volumes de dados são mantidos por longos períodos. O SQLite, por exemplo, apresentou uso considerável de RAM devido à manipulação de arquivos e tabelas, o que limita sua escalabilidade em dispositivos com baixa capacidade. Em contrapartida, permite armazenar registros estruturados de forma eficiente.

O LittleFS se destacou por sua leveza, ocupando pouco espaço no flash e sendo capaz de lidar com pequenos arquivos de forma eficiente, embora possua quantidade total limitada de dados devido ao tamanho da memória interna. Já o microSD oferece maior capacidade de armazenamento, ideal para aplicações que requerem registro de dados em grande escala. No entanto, o uso do cartão adiciona consumo de energia e ocupa pinos de comunicação SPI.

O próximo quadro resume o consumo de memória e armazenamento das abordagens testadas:

Método de Armazenamento	Ocupação de RAM	Uso de Memória Flash	Capacidade de Armazenamento	Observação
Firebase	Baixa	Alta (bibliotecas)	Ilimitada (nuvem)	Necessita Wi-Fi constante.
InfluxDB	Baixa	Alta (bibliotecas)	Ilimitada (nuvem)	Ideal para séries temporais.
SQLite	Média	Média	Limitada à flash local	Boa estruturação de dados.
LittleFS	Baixa	Baixa	Limitada (interna)	Leve e eficiente para pequenos volumes.
microSD	Média	Média	Alta (depende do cartão)	Requer componente externo.

Quadro 6 - Comparativo do consumo de memória e espaço de armazenamento nas diferentes abordagens de persistência de dados com ESP32
Fonte: Elaborado pelo autor (2025)

Na próxima subseção, será analisada a latência de transmissão e acesso dos dados, considerando os tempos de resposta e sincronização nos diferentes métodos de armazenamento.

4.3 Latência de transmissão e acesso

A latência representa o tempo total entre o envio de uma informação por um dispositivo e o momento em que ela está acessível para uso ou análise. Em sistemas de IoT, essa métrica é especialmente relevante em aplicações que demandam resposta rápida ou operações em tempo real, como automação industrial, monitoramento médico e controle de trânsito.

Nos testes realizados, cada método de armazenamento foi avaliado quanto à latência total de transmissão e acesso. Para bancos em nuvem (Firebase e InfluxDB), considerou-se o tempo desde o envio da mensagem MQTT ou HTTP até a sua disponibilização na interface de leitura. Para bancos locais (SQLite, LittleFS e microSD), foi mensurado o tempo entre a coleta dos dados e sua gravação com sucesso no meio de armazenamento, seguido da leitura imediata.

Os resultados mostraram que os métodos locais apresentam latências significativamente menores. O LittleFS, por exemplo, alcançou latências abaixo de 30 ms para gravação e leitura, o que o torna ideal para aplicações embarcadas que

requerem resposta quase imediata. O microSD apresentou latência levemente superior, em torno de 40 a 60 ms, enquanto o SQLite oscilou entre 50 e 70 ms, dependendo da estrutura de consulta.

Em contraste, os métodos em nuvem sofreram maior variação conforme a qualidade da conexão de internet. O Firebase, por exemplo, apresentou latências que variaram entre 400 e 900 ms, enquanto o InfluxDB variou entre 300 e 700 ms, sendo mais eficiente quando hospedado em servidores próximos geograficamente ao dispositivo.

O próximo quadro resume os tempos médios observados nos experimentos:

Método de Armazenamento	**Latência de Transmissão e Acesso (ms)**
Firebase	400 - 900
InfluxDB	300 - 700
SQLite	50 - 70
LittleFS	20 - 30
microSD	40 - 60

Quadro 7 - Comparativo de latência de transmissão e acesso entre diferentes métodos de armazenamento em IoT
Fonte: Elaborado pelo autor (2025)

Com base nesses dados, conclui-se que, para sistemas com exigência de baixa latência, os métodos locais são mais indicados. Por outro lado, aplicações com foco em escalabilidade e acessibilidade remota podem aceitar a latência maior dos serviços em nuvem. Na próxima subseção, será abordada a robustez e a persistência dos dados em diferentes cenários de falhas.

4.4 Robustez e persistência dos dados

A robustez e a persistência dos dados em sistemas de IoT referem-se à capacidade dos métodos de armazenamento de manter a integridade das informações mesmo diante de interrupções inesperadas, como quedas de energia, falhas de conexão ou reinicializações do dispositivo. Esses atributos são essenciais para garantir a confiabilidade dos dados em aplicações críticas, especialmente em ambientes remotos ou com infraestrutura limitada.

Durante os experimentos realizados, foi possível observar diferentes níveis de robustez e persistência entre os métodos analisados. Os métodos locais, como microSD e LittleFS, apresentaram um bom desempenho desde que implementados com as devidas precauções. Embora o uso do microSD ofereça ampla capacidade de armazenamento, ele é sensível a falhas de gravação em casos de queda de energia durante a escrita. Por outro lado, o LittleFS demonstrou maior segurança nesse aspecto, pois utiliza mecanismos de *journaling* que reduzem significativamente o risco de corrupção dos dados.

O SQLite, mesmo sendo um banco local, demonstrou alto nível de robustez devido ao seu modelo transacional. Isso significa que as operações de escrita só são concluídas se forem totalmente bem-sucedidas, o que evita registros incompletos e aumenta a persistência dos dados mesmo após interrupções abruptas.

Em contrapartida, os métodos em nuvem, como Firebase e InfluxDB, dependem fortemente da conectividade para garantir a persistência. Quando há conexão estável, esses serviços oferecem alta confiabilidade, com redundância de dados, replicação em servidores e backups automáticos. No entanto, na ausência de internet, os dados podem ser perdidos se não houver uma estratégia local de buffer implementada.

O quadro a seguir apresenta um resumo comparativo entre os métodos analisados:

Método de Armazenamento	**Robustez em Casos de Falha**	**Persistência dos Dados**
Firebase	Alta (com internet)	Alta (com redundância)
InfluxDB	Alta (com internet)	Alta
SQLite	Alta	Alta
LittleFS	Média-Alta (com cuidados)	Média-Alta
microSD	Média (risco de corrupção)	Média

Quadro 8 - Comparativo de robustez e persistência dos dados em diferentes métodos de armazenamento
Fonte: Elaborado pelo autor (2025)

Para finalizarmos, conclui-se que, para aplicações críticas, é recomendável combinar métodos de armazenamento locais e em nuvem. O armazenamento local pode servir como um backup temporário até que os dados sejam transmitidos com sucesso para a nuvem, garantindo, assim, maior confiabilidade e segurança das informações em qualquer condição de operação.

FIAP

REFERÊNCIAS

FIREBASE. **Firebase Realtime Database**. Disponível em: <https://firebase.google.com/?hl=pt-br>. Acesso em: 4 maio 2025.

INFLUXDATA. **InfluxDB Cloud – Data Explorer**. Disponível em: <https://us-east-1-1.aws.cloud2.influxdata.com/orgs/e7b0d10e7a48d718/data-explorer?fluxScriptEditor>. Acesso em: 26 jun. 2025.

MISCHIANTI, Renzo. ESP32 DevKitC V4 high resolution pinout and specs. **Mischianti,** 2021. Disponível em: <https://mischianti.org/esp32-devkitc-v4-high-resolution-pinout-and-specs/>. Acesso em: 26 jun. 2025.

WOKWI. Simulador online de circuitos e microcontroladores. **Wokwi**, 2025. Disponível em: <https://wokwi.com/>. Acesso em: 26 jun. 2025.